

1 KNN

1.1 The Very Basics

Learning: find a model f that maps from inputs to outputs given a labeled training dataset.
Inference: use a model f to predict an output y given a new example $\vec{x}$
A model is a function f mapping input $\vec{x}$ to output prediction y .
A model family is a set of models that share the same structure but differ in parameters (e.g. layers in a neural network)
Accuracy is calculated as follows:

$$\frac{\sum_{i=1}^N \mathbb{I}\left[t^{(i)} = y^{(i)}\right]}{N}$$

A **parameter** is the internal variables of a model learned from data whilst training.
A **hyperparameter** are the settings chosen before training that controls how the learning process works.
Train: fit parameters.
Val: tune hyperparameters / pick model.
Test: *When:* use once after model & hyperparameters are chosen. *Purpose:* get an unbiased estimate of generalization error. *If used for tuning:* test info leaks into model selection, so the test error becomes optimistically biased.
Regression vs classification is decided by the *output* only (continuous vs discrete).
 t = target, y = prediction; $x_j^{(i)}$ = feature j of example i .
100% training accuracy $\nrightarrow$ good test accuracy; May overfit.
Data augmentation: changing the training data by a little bit to create new training data (shift/rotate).
Parametric models fix the parameter count. Non-parametric models don't. $\mathcal{O}(1)$ space complexity with respect to the feature count, and never $\Omega(N)$. Hyperparameters are out of this scope.

1.2 KNN

1NN: predict $t^{(c)}$, where

$$\vec{x}^{(c)} = \arg \min_{\vec{x}^{(i)} \in \text{training set}} \text{distance}(\vec{x}^{(i)}, \vec{x})$$

kNN: predict using the majority class among the k closest training examples to $\mathbf{x}$.

Distances:

$$L_2: \sqrt{\sum_j (a_j - b_j)^2} \quad \left| \begin{array}{l} \text{Cos: } \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \\ L_p: (\sum_j |a_j - b_j|^p)^{1/p} \quad \left| \begin{array}{l} L_\infty: \max_j |a_j - b_j| \end{array} \right. \right.$$

Cos: ignores vector length, only direction $\rightarrow$ good for text.
Normalize: features to mean = 0 and variance = 1,

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x} - \mu}{\sigma}$$

Rule: compute μ_j, σ_j from the training set only; Reuse them for validation/test (else leakage).
Boundary (1NN):
Euclidean: (1) for nearby pairs of different classes draw the perpendicular bisector ($\mathbf{b} - \mathbf{a}$)^T $\mathbf{x} = \frac{\|\mathbf{b}\|^2 - \|\mathbf{a}\|^2}{2}$; (2) cut bisectors at Voronoi vertices (points equidistant to 3 training points); (3) keep only edges separating different classes; (4) check with a test point.
Manhattan: bisector of 2 points depends on their bounding box: wide ($|\Delta x_1| > |\Delta x_2|$) $\rightarrow$ 45° segment inside the box, then vertical rays; Tall $\rightarrow$ 45° segment, then horizontal rays; Square $\rightarrow$ diagonal + 2-D tie regions (state the tie-breaking rule).
Cosine: only the angle matters $\rightarrow$ regions are wedges from the origin; Boundary = rays along angle bisectors of adjacent points of different classes.
Pros: simple; No training; Works for regression and classification; Adapts easily to new data.
Cons:
(1) **Curse of dimensionality:** in high dimensions most points are far apart from each other (L_∞ on $[0, 1]^d$: fraction within ϵ of origin = ϵ^d ; $\therefore \epsilon < 1, d \uparrow \Rightarrow \epsilon^d \rightarrow 0$).
(2) Sensitive to ranges of features (larger range dominates distance calculations).
(3) Slow prediction: distance to all N points, $\mathcal{O}(ND)$ per query.
(4) Must store the entire training set, $\mathcal{O}(ND)$ space.
Choose k :
 $k \uparrow \rightarrow$ boundary smoother, less sensitive to individual points $\rightarrow$ underfit; $k = N \rightarrow$ always predicts the majority class.
 $k = 1 \rightarrow$ sharp boundary, overfit; 100% training accuracy (unless duplicate $\mathbf{x}$ with different labels).
No single best k , depends on N ; Heuristic $k < \sqrt{N}$ ($k \rightarrow \infty, k/N \rightarrow 0$).
Pick k (any hyperparameter) via validation set or cross-validation.
Workflow:
(1) Split data into train/validation/test.
(2) For each K : fit KNN on train.
(3) Compute accuracy of each on validation.
(4) Pick the K with the best validation accuracy.
(5) Report the test accuracy of that K .

2 Decision Trees

Tree size: large $\rightarrow$ overfit; Small $\rightarrow$ underfit.
Inference: trace the tree from root to leaf.
Entropy:

$$H(Y) = -\sum_y p(y) \log_2 p(y)$$

Conditional entropy:

$$H(Y | X = x) = -\sum_y p(y | x) \log_2 p(y | x)$$

Expected conditional entropy:

$$H(Y | X) = \sum_x p(x) H(Y | X = x)$$

Information gain:

$$IG(Y | X) = H(Y) - H(Y | X)$$

IG table (Y = label/class, X = split on a feature; count examples per split side & class):

	Class 1	Class 2	Total
Split cond. 1	a	b	$a + b$
Split cond. 2	c	d	$c + d$
Total	$a + c$	$b + d$	N

Use: $H(Y)$ from the Total row; $H(Y | X)$ from the two split rows:

$$H(Y | X) = \frac{a+b}{N} H\left(\frac{a}{a+b}, \frac{b}{a+b}\right) + \frac{c+d}{N} H\left(\frac{c}{c+d}, \frac{d}{c+d}\right)$$

Properties:
 $H(Y) \geq 0$; Certain outcome $\rightarrow H = 0$; Binary: $\max H = 1$ bit at $p = 0.5$.
 $H(Y | X) \leq H(Y)$ (i.e. $IG \geq 0$).
 Y, X independent $\Rightarrow IG(Y | X) = 0$ (i.e. $H(Y | X) = H(Y)$).
 Y fully determined by $X \Rightarrow H(Y | X) = 0 \Rightarrow IG(Y | X) = H(Y)$.

2.1 Decision Tree Algorithm

Stop splitting when:
(1) All examples have the same label.
(2) No features left.
(3) Max depth reached (hyperparameter).
(4) Loss decreases too little (hyperparameter).
(5) Node has fewer than a min number of samples (hyperparameter).
Split: pick the split with max IG (not training error, $\therefore$ it ignores the uncertainty left in the child nodes).
Leaf prediction: majority class.
Properties:
Greedy $\rightarrow$ not globally optimal ($\therefore$ finding the optimal tree is NP-hard).
Boundaries are axis-aligned (each split tests $x_j \leq \theta$).
Continuous feature: only $\leq N-1$ thresholds to check (mid-points between sorted values).
Scale-invariant.
Boundary:
(1) Each split $x_j \leq \theta$ is a vertical/horizontal line, drawn only inside its parent node's region.
(2) Leaves = axis-aligned rectangles.
(3) Label each rectangle with its leaf's prediction (majority class).
Train cost vs KNN: DT more ($\therefore$ compute IG of each split to choose the best feature & threshold); KNN less (no computation, just stores data).

3 Linear / GD

Setup: N = number of examples, D = number of input features.
Dummy: $x_0^{(i)} = 1 \Rightarrow b = w_0 x_0^{(i)}$:
 $\mathbf{x}^{(i)} = [1, x_1^{(i)}, \dots, x_D^{(i)}]^T \in \mathbb{R}^{(D+1) \times 1}$
 $\mathbf{w} = [w_0, \dots, w_D]^T \in \mathbb{R}^{(D+1) \times 1}$
 $y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} \in \mathbb{R}$
$$X = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \vdots \\ \mathbf{x}^{(N)T} \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \text{ (row = example)}$$

$\mathbf{t}, \mathbf{y} \in \mathbb{R}^{N \times 1}, \mathbf{y} = X\mathbf{w}$
Prediction: $y = \mathbf{w}^T \mathbf{x} + b, \mathbf{y} = X\mathbf{w}$ ($X \in \mathbb{R}^{N \times (D+1)}$, row = example).
Gradient direction: ∇ = direction of steepest ascent, perpendicular to contours $\rightarrow$ GD steps along $-\nabla$.
Learning rate α : too big $\rightarrow$ overshoot/diverge; Too small $\rightarrow$ converges slowly.
Feature map: $\psi(\mathbf{x})$ (e.g. $[1, x, x^2]$) $\rightarrow$ fits nonlinear functions of $\mathbf{x}$, model still linear in $\mathbf{w}$.
Cost function: $\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y^{(i)}, t^{(i)})$
Loss function: $\mathcal{L}(y, t) = \frac{1}{2} (y - t)^2$
Loss = how off one single example is; Cost = average over a set of examples.
Linear cost:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N \left(\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)} \right)^2$$
$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} (\mathbf{X}\mathbf{w} - \mathbf{t})^T (\mathbf{X}\mathbf{w} - \mathbf{t})$$

3.1 Gradient of the Cost Function

$\mathcal{L}(y^{(i)}, t^{(i)}) = \frac{1}{2} (y^{(i)} - t^{(i)})^2 \in \mathbb{R}$
 $\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_i \mathcal{L} \in \mathbb{R}$
Let $\mathbf{e} = \mathbf{y} - \mathbf{t} \in \mathbb{R}^{N \times 1}$: $\mathbf{e}^T \mathbf{e} = \sum_i (y^{(i)} - t^{(i)})^2$
$$\Rightarrow \mathcal{E} = \frac{1}{2N} \mathbf{e}^T \mathbf{e} = \frac{1}{2N} (\mathbf{X}\mathbf{w} - \mathbf{t})^T (\mathbf{X}\mathbf{w} - \mathbf{t}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|_2^2$$

Gradient:
 $\frac{\partial \mathcal{E}}{\partial y^{(i)}} = y^{(i)} - t^{(i)}, \quad \frac{\partial y^{(i)}}{\partial w_j} = x_j^{(i)}$
 $\Rightarrow \frac{\partial \mathcal{E}}{\partial w_j} = (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$
 $\Rightarrow \frac{\partial \mathcal{E}}{\partial \mathbf{w}_j} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$
 $\Rightarrow \nabla_{\mathbf{w}} \mathcal{L} = \frac{(\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) \mathbf{x}^{(i)}}{N} \in \mathbb{R}^{D+1}$
 $\Rightarrow \nabla_{\mathbf{w}} \mathcal{E} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$
Let $\mathbf{r}^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)} \in \mathbb{R}, \mathbf{r} = \mathbf{X}\mathbf{w} - \mathbf{t} \in \mathbb{R}^{N \times 1}$:
$$\nabla_{\mathbf{w}} \mathcal{E} = \frac{1}{N} \sum_i r^{(i)} \mathbf{x}^{(i)} = \frac{1}{N} X^T \mathbf{r} = \frac{1}{N} X^T (\mathbf{X}\mathbf{w} - \mathbf{t}) \in \mathbb{R}^{D+1}$$

KEY: $\sum_i r^{(i)} \mathbf{x}^{(i)} = X^T \mathbf{r}$
($\therefore$ columns of X^T are $\mathbf{x}^{(i)}$, weighted by $r^{(i)}$)
Solve:
(1) **Direct (closed form):** $\nabla \mathcal{E} = 0 \Rightarrow X^T X \mathbf{w} = X^T \mathbf{t} \Rightarrow \mathbf{w} = (X^T X)^{-1} X^T \mathbf{t}$
Pros: closed form, exact & provably optimal, no iteration.
Cons: closed-form solution may not exist for most problems; Can't generalize to most models; Computationally expensive: $(X^T X)^{-1}$ is $\mathcal{O}(D^3)$.
(2) **GD:**
$$\underbrace{\mathbf{w}}_{\in \mathbb{R}^{(D+1) \times 1}} \leftarrow \mathbf{w} - \underbrace{\frac{\alpha}{N} X^T (\mathbf{X}\mathbf{w} - \mathbf{t})}_{\in \mathbb{R}^{(D+1) \times 1}}$$

Large Weight: *Why problematic:* one feature contributes a lot to the prediction, but a good prediction uses many features $\rightarrow$ sign of overfitting.
Regularization:
Def: modify training to choose models using criteria beyond training error, i.e. minimize $\mathcal{E} + \lambda \mathcal{R}$ (penalize large $\mathbf{w}$).
Effect of λ : small $\rightarrow$ overfit; Big $\rightarrow$ underfit.
Not: forcing weights to similar scale; Not guaranteeing 0 training error; Adds (not removes) hyperparameter λ .

$$\Delta \frac{1}{2} \sum_{j=0}^D w_j^2$$

∇ regularizer:

$$\frac{\partial}{\partial w_j} \left(\frac{\lambda}{2} \sum_{j=0}^D w_j^2 \right) = \lambda w_j$$
$$\nabla_{\mathbf{w}} \left(\frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \right) = \lambda \mathbf{w}$$

With bias term:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} (\mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{t})^T (\mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{t})$$
$$\nabla_{\mathbf{w}} \mathcal{E}(\mathbf{w}) = \frac{1}{N} X^T (\mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{t})$$
$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N \left(\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)} \right) x_j^{(i)}$$
$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial b} = \frac{1}{N} (\mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{t})$$

3.2 Stochastic GD

SGD update (i random):

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{E}}{\partial \mathbf{w}}$$

Iteration: 1 weight update from 1 mini-batch of B examples (B = hyperparameter).
Epoch: every training example used once ($= N/B$ iterations).

	Batch	Mini-batch	SGD
#examples/update	all N	$B, 1 < B < N$	1
Gradient accuracy	exact	noisy ($\downarrow$ as $B \uparrow$)	noisiest
Cost/update	$\mathcal{O}(N)$	$\mathcal{O}(B)$	$\mathcal{O}(1)$
Vectorization	high (memory limit?)	high, fits hardware	low

Mini-batch gradient: unbiased estimate of the full gradient.
SGD Pros: noisy gradient $\rightarrow$ can escape flat regions (e.g. plateaus); No guarantee of reaching the global optimum.
Ravine:
Def: cost is steep in some dimensions, flat in others (gradient magnitudes differ a lot across dimensions).
Effect: GD oscillates in steep dimensions, moves slowly in flat dimensions.
Cause: features on very different scales.
Fix: normalize features.

3.3 Vectorizing Tips

$X \in \mathbb{R}^{N \times D}$; Other variables' shapes are inferred.
Answer format: vectors are columns; For every new symbol state its dimension and define its components (e.g. $X \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{x}_i^T$).
Dot product:
$$\sum_{i=1}^N u_i v_i = \mathbf{u}^T \mathbf{v}$$

Hadamard product:
$$\mathbf{u} \odot \mathbf{v} = \text{diag}(\mathbf{u}) \mathbf{v} = \left(\mathbf{u}^T \text{diag}(\mathbf{v}) \right)^T$$

Sum a vector:
$$\mathbf{u}^T \mathbf{1} = \sum_{i=1}^N u_i$$

Sum axis 1: horizontal squish $\rightarrow$ $< (X1 \in \mathbb{R}^{N \times 1})$
$$X1$$

Sum axis 0: vertical squish $\downarrow$ ($1^T X \in \mathbb{R}^{1 \times D}$)
$$1^T X$$

May need to transpose.
Weighted sum axis 1: weighted horizontal squish ($M \in \mathbb{R}^{P \times Q}$, column $j = \mathbf{m}_j \in \mathbb{R}^{P \times 1}$, weights $\mathbf{u} \in \mathbb{R}^{Q \times 1}$)
$$\sum_{j=1}^Q u_j \mathbf{m}_j = M \mathbf{u} \in \mathbb{R}^{P \times 1}$$

Weighted sum axis 0: weighted vertical squish ($M \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{m}_i^T \in \mathbb{R}^{Q \times 1}$, weights $\mathbf{u} \in \mathbb{R}^{P \times 1}$)
$$\sum_{i=1}^P u_i \mathbf{m}_i^T = \mathbf{u}^T M \in \mathbb{R}^{1 \times Q}$$

Transpose of above (column form):
$$\sum_{i=1}^P u_i \mathbf{m}_i = M^T \mathbf{u} \in \mathbb{R}^{Q \times 1}$$

Row-wise multiplication:
$$\text{diag}(\mathbf{u}) X = \begin{bmatrix} - & u_1 \mathbf{x}^{(1)} & - \\ - & u_i \mathbf{x}^{(i)} & - \\ - & u_N \mathbf{x}^{(N)} & - \end{bmatrix}$$

Column-wise multiplication:
$$X \cdot \text{diag}(\mathbf{v}) = \begin{bmatrix} | & | & | \\ v_1 \mathbf{x}_1 & v_2 \mathbf{x}_2 & v_3 \mathbf{x}_3 \\ | & | & | \end{bmatrix}$$

Tip: output height = height of leftmost matrix (e.g. $X^T \mathbf{r} \in \mathbb{R}^{D \times 1}$).
$$\sum_{i=1}^N r^{(i)} \mathbf{x}^{(i)} = X^T \mathbf{r}$$

Shape reasoning: the summed index (N) must be the inner dimension so it cancels; The first dimension of the result goes on the left.
Stack predictions: $(X\mathbf{w})_i = \mathbf{w}^T \mathbf{x}^{(i)}, X\mathbf{w} \in \mathbb{R}^{N \times 1}$.
Scalar with vector ($s \in \mathbb{R}$):
$$\alpha_i = u_i + s \Rightarrow \boldsymbol{\alpha} = \mathbf{u} + s \mathbf{1}$$
$$\beta_i = s u_i \Rightarrow \boldsymbol{\beta} = s \mathbf{u}$$

Outer product:
$$M_{ij} = u_i v_j \Rightarrow M = \mathbf{u} \mathbf{v}^T \in \mathbb{R}^{P \times Q}$$

Output type: inner product $\rightarrow$ scalar; Element-wise $\rightarrow$ vector; Outer $\rightarrow$ matrix.
Stack of dot products ($M \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{m}_i^T$; $\mathbf{u} \in \mathbb{R}^{Q \times 1}$):
$$\alpha_i = \mathbf{m}_i^T \mathbf{u} \Rightarrow \boldsymbol{\alpha} = M \mathbf{u} \in \mathbb{R}^{P \times 1}$$

Two weights per term ($M \in \mathbb{R}^{P \times Q}$, row $i = \mathbf{m}_i^T$; $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{P \times 1}$):

$$\sum_i u_i v_i \mathbf{m}_i = M^T (\mathbf{u} \odot \mathbf{v}) = M^T \text{diag}(\mathbf{u}) \mathbf{v} \in \mathbb{R}^{Q \times 1}$$

Double sums:

$$\begin{aligned} \sum_i \sum_j u_i v_j m_{ij} &= \mathbf{u}^T M \mathbf{v} \\ \sum_i \sum_j u_i v_j m_{ij}^2 &= \mathbf{u}^T (M \odot M) \mathbf{v} \\ \sum_i \sum_k u_i m_{ik} u_k &= \mathbf{u}^T M \mathbf{u} \end{aligned}$$

Pitfall: $\mathbf{a} \odot \mathbf{b} \neq \mathbf{a} \text{diag}(\mathbf{b}) - \mathbf{a} \in \mathbb{R}^{P \times 1}$ times $\text{diag}(\mathbf{b}) \in \mathbb{R}^{P \times P}$ is undefined; Use $\text{diag}(\mathbf{b})\mathbf{a}$.

Pitfall: $\frac{1}{N} (X\mathbf{w} - \mathbf{t})^T X \in \mathbb{R}^{1 \times (D+1)}$ is a row; Gradient must be a column $\Rightarrow \frac{1}{N} X^T (X\mathbf{w} - \mathbf{t})$.

Variants of linear regression gradient ($\mathbf{r} = X\mathbf{w} - \mathbf{t}$):

- Per-weight λ_j : $\frac{1}{N} X^T \mathbf{r} + \lambda \odot \mathbf{w} = \frac{1}{N} X^T \mathbf{r} + \text{diag}(\lambda) \mathbf{w}$
- Per-example weight α_i : $\sum_i \alpha_i \mathbf{r}^{(i)} \mathbf{x}^{(i)} = X^T \text{diag}(\mathbf{a}) \mathbf{r}$
- Per-feature scale α_j : predictions $X \text{diag}(\boldsymbol{\alpha}) \mathbf{w}$, gradient gains a $\text{diag}(\boldsymbol{\alpha})$ on the left: $\frac{1}{N} \text{diag}(\boldsymbol{\alpha}) X^T \mathbf{r}'$, $\mathbf{r}' = X \text{diag}(\boldsymbol{\alpha}) \mathbf{w} - \mathbf{t}$

Absorb bias: $\mathbf{w}_1^T \mathbf{x}_1 + b = \mathbf{w}_2^T \mathbf{x}_2$ with $\mathbf{w}_2 = \begin{bmatrix} b \\ \mathbf{w}_1 \end{bmatrix}$,

$$\mathbf{x}_2 = \begin{bmatrix} 1 \\ \mathbf{x}_1 \end{bmatrix} \quad (\mathbf{w}_2, \mathbf{x}_2 \in \mathbb{R}^{(D+1) \times 1}).$$

4 Logistic Regression

Output: $y = \sigma(\mathbf{w}^T \mathbf{x}) = p(t = 1 \mid \mathbf{x})$, threshold 0.5.

Linear model:

$$z = \mathbf{w}^T \mathbf{x}$$

Prediction:

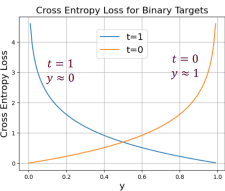
$$y = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Loss functions:

- Zero-One Loss:** $L_{0-1}(y, t) = \mathbb{I}[y \neq t]$
 - Non-differentiable / zero gradient $\rightarrow$ can't update $\mathbf{w}$.
- Squared Error Loss:** $L_{SE}(z, t) = \frac{1}{2} (z - t)^2$
 - Penalizes confident correct predictions.
- Sigmoid + Squared Error:**
 - $y = \sigma(z) = \frac{1}{1+e^{-z}}$, $L_{SE}(y, t) = \frac{1}{2} (y - t)^2$
 - Vanishing gradient when the prediction is very wrong.
- Cross entropy:** $\begin{cases} -\log(y) & t = 1 \\ -\log(1 - y) & t = 0 \end{cases}$

Cross entropy:

$$-t \log(y) - (1 - t) \log(1 - y)$$



4.1 Cross-Entropy Loss Equations

Linear model:

$$z = \mathbf{w}^T \mathbf{x}$$

Classification:

$$\begin{aligned} y &= \sigma(z) = \frac{1}{1 + e^{-z}} \\ \sigma'(z) &= \sigma(z) (1 - \sigma(z)) \\ \frac{\partial y}{\partial z} &= y(1 - y) \end{aligned}$$

Loss:

$$\mathcal{L}_{CE}(y, t) = -t \log(y) - (1 - t) \log(1 - y)$$

Gradient:

Where $\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{CE}(y^{(i)}, t^{(i)})$

$$\begin{aligned} \frac{\partial \mathcal{L}_{CE}}{\partial y} &= -\frac{t}{y} + \frac{1-t}{1-y} \\ &= \frac{-t(1-y) + (1-t)y}{y(1-y)} = \frac{y-t}{y(1-y)} \end{aligned}$$

$$\begin{aligned} \frac{\partial y}{\partial z} &= y(1-y), & \frac{\partial z}{\partial w_j} &= x_j \\ \frac{\partial \mathcal{L}_{CE}}{\partial w_j} &= \frac{\partial \mathcal{L}_{CE}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_j} \\ &= \left(-\frac{t}{y} + \frac{1-t}{1-y} \right) \cdot y(1-y) \cdot x_j \\ &= \frac{y-t}{y(1-y)} \cdot y(1-y) \cdot x_j \\ &= (y-t)x_j \end{aligned}$$

GD update for LogReg:

$$\begin{aligned} w_j &\leftarrow w_j - \alpha \frac{\partial \mathcal{E}}{\partial w_j} \\ &= w_j - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)} \\ \mathbf{w} &\leftarrow \mathbf{w} - \frac{\alpha}{N} X^T (\sigma(X\mathbf{w}) - \mathbf{t}) \end{aligned}$$

Pros:

$\mathcal{L} \rightarrow \infty$ for confident wrong predictions.

$\partial \mathcal{L} / \partial z = y - t$ (no saturation).

Convex in $\mathbf{w}$; Maximum likelihood.

Sigmoid+SE Cons: $\partial \mathcal{L} / \partial z = (y - t)y(1 - y) \approx 0$ when the prediction is very wrong.

Boundary (threshold a : predict 1 iff $y \geq a$):

$$\begin{aligned} \sigma(z) = a &\iff 1 + e^{-z} = \frac{1}{a} \iff e^{-z} = \frac{1-a}{a} \\ &\iff z = \mathbf{w}^T \mathbf{x} = \ln \frac{a}{1-a} \end{aligned}$$

Hyperplane; Class 1 where $z \geq \ln \frac{a}{1-a}$ ($\therefore \sigma$ increasing).

If $a = 0.5$: $\ln \frac{0.5}{0.5} = \ln 1 = 0 \Rightarrow$ boundary $\mathbf{w}^T \mathbf{x} = 0$; Class 1 where $z \geq 0$.

Steps:

- Write $z = \mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2$ (with $x_0 = 1$).
- Set $z = \ln \frac{a}{1-a}$ ($= 0$ if $a = 0.5$) and solve for $x_2 \rightarrow$ boundary line.
- Plug a test point (e.g. origin) into z : the side with $z > \ln \frac{a}{1-a}$ is class 1, the other side is class 0.
- Draw the line through two easy points (intercepts) and label both regions.

4.2 LogReg vs kNN

Parametric: hypothesis space characterized by a fixed-size set of parameters (e.g. LogReg).

Non-parametric: complexity can grow with the amount of training data (e.g. kNN).

Pros (vs kNN):

Storage: only $D+1$ weights, training data discarded after fitting (parametric).

Prediction: fast, one dot product $\mathbf{w}^T \mathbf{x}$ with fixed learned weights, $O(D)$.

Interpretability: weights rank the features.

Generalization: good from limited data if the linear assumption holds.

Cons:

Inductive bias: linear decision boundary $\rightarrow$ underfits a nonlinear boundary.

Fixed number of parameters $\rightarrow$ complexity doesn't grow with more data (unlike kNN).

	LogReg	kNN
Model type	Parametric	Non-parametric
Inductive bias	Linear decision boundary	Nearby inputs share labels
#Parameters	$D+1$ weights	Grows with the data
Storage	$D+1$ weights	Entire training set
Prediction cost	One dot product	Distance to every example
Interpretability	Weights rank the features	Neighbours explain one prediction
Typical failure	Underfits a nonlinear boundary	Overfits when k is small

4.3 Softmax Regression ($K \geq 3$)

Target: one-hot $\mathbf{t} \in \{0, 1\}^K$ (1 at true class). Why not integers: imply order + distance, renumbering changes model.

Why one-hot: no order, all pairs equidistant.

Logits: $\mathbf{z} = W\mathbf{x} \in \mathbb{R}^K$, $W \in \mathbb{R}^{K \times (D+1)}$, row $k = [b_k, \mathbf{w}_k^T]$,

$z_k = \mathbf{w}_k^T \mathbf{x}$ (unnormalized). Predict $\arg \max_k z_k$.

$$y_k = \frac{\exp(z_k)}{\sum_{k'=1}^K \exp(z_{k'})}, \quad \mathbf{y} = \text{softmax}(\mathbf{z}) \in \mathbb{R}^K$$

Properties:

- Probability distribution: $y_k \in (0, 1)$, $\sum_k y_k = 1$.
- Order-preserving ($\cdot \exp \uparrow$) $\Rightarrow \arg \max y = \arg \max z$, softmax doesn't change the boundary; $y_k =$ confidence.
- Smooth, differentiable approximation of argmax $\Rightarrow$ GD works.
- $K = 2$: $y_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = \sigma(z_1 - z_2)$, $y_2 = \sigma(-z)$.

Loss: $\mathcal{L}_{CE} = -\sum_k t_k \log y_k = -\mathbf{t}^T \log \mathbf{y} = -\log y_{\text{true}}$ (1 nonzero term).

Gradient: $\frac{\partial \mathcal{L}}{\partial z_k} = y_k - t_k$, $\nabla_{\mathbf{w}_k} \mathcal{L} = (y_k - t_k) \mathbf{x}$

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \frac{\alpha}{N} \sum_i (y_k^{(i)} - t_k^{(i)}) \mathbf{x}^{(i)}$$

Matrix: $\nabla_W \mathcal{E} = \frac{1}{N} (Y - T)^T X$ ($Y, T \in \mathbb{R}^{N \times K}$, rows $\mathbf{y}^{(i)T}, \mathbf{t}^{(i)T}$).

Boundary i/j : needs BOTH, (1) $z_i = z_j \iff (\mathbf{w}_i - \mathbf{w}_j)^T \mathbf{x} = 0$ (line); (2) $z_k \leq z_i \forall k \neq i, j$.

Steps: solve (1) for line $\rightarrow$ substitute into (2) $\rightarrow$ inequality gives ray/segment $\rightarrow$ all 3 lines meet where $z_1 = z_2 = z_3 \rightarrow$ test a point per region. At most $K(K-1)/2$ boundaries.

Cons: linear boundaries only (convex regions); Can't do nonlinear.